

Rival Hunt

Unity Gameplay Systems Assessment

GitHub: github.com/ET-51/rival-hunt-unity

Playable build: github.com/ET-51/rival-hunt-unity/releases/latest

Inspiration and twist

Rival Hunt is inspired by the gem collection and leveling loop in Vampire Survivors. The twist is a simulated Rival that races the player through the same XP economy. Collecting gems increases the player's level and temporarily slows the Rival, so a normal pickup becomes both progress and defense.

Gameplay summary

The player moves through a top-down 2D arena using WASD or the arrow keys. Gems spawn inside the arena and grant XP on contact. The player and Rival each use a level and XP bar. This assessment build uses a one-minute timer so the complete system can be reviewed quickly. When the timer reaches zero, the higher level wins.

- Starving: gems collected during the last 10 seconds reduce the Rival's rate by 8% each, capped at 40%.
- Rubber-band: a side that falls three levels behind receives a 15% XP boost until the gap closes.
- Final Sprint: all XP gains are doubled during the final 15 seconds.
- Result: the final levels are compared; equal levels produce a tie.

Architecture and data flow

Script	Responsibility
PlayerController	Reads movement input and applies Rigidbody2D velocity.
Gem	Handles pickup, magnet movement, XP delivery, and the collection event.
GemSpawner	Spawns gems inside arena bounds and enforces an alive limit.
XPSystem	Reusable XP and leveling component used by both racers.
RivalSimulator	Combines base rate, starving, rubber-band, and sprint modifiers.
MatchController	Owns the timer, match state, player multipliers, and result.
RaceUI	Displays race feedback and listens to gameplay events.

Main flow: Gem pickup -> player XPSystem -> level progress. The pickup event also reaches RivalSimulator, which records recent gems and changes the Rival rate. MatchController announces state changes, while RaceUI displays them without owning gameplay rules.

Key decisions

The Rival is simulated

The assessment focuses on the race system, so the Rival is a number rather than a world-space character. This removes pathfinding and combat from the scope and makes the mechanic easy to balance through one base XP rate and three readable multipliers.

One XPSystem serves both sides

The player and Rival follow the same leveling rules. Reusing one XPSystem avoids duplicating level calculations and guarantees that both sides expose the same level and progress values to the UI.

Match state belongs in MatchController

Running, FinalSprint, and Ended are represented by a small enum. Keeping transitions in one controller prevents the timer, UI, and Rival from each deciding the current state. Other scripts read the state or listen to events.

How to run and test

- Open the project in Unity 6000.3.19f1 and load Assets/Scenes/Main.unity.
- Press Play and use WASD or arrow keys to collect gems.
- Collect quickly to see the Rival slow, then stop to let it recover.
- At 0:15, confirm that the Final Sprint banner and timer color appear.
- At 0:00, confirm that the correct result and both levels are shown.
- Press Play Again to reload the match.

Known limitations

The project uses a small set of original pixel-art assets and one arena because the focus is the gameplay system. The Rival has no physical character by design; its thin HUD strip and moving portrait communicate the race. Enemies, weapons, upgrades, save data, and extra menus are outside the chosen scope.

What I would improve next

With more time, I would run repeated playtests, tune several Rival difficulty presets, add automated tests for XP thresholds and match-state transitions, and improve accessibility for the lead feedback. I would validate the race mechanic before adding combat or a larger content layer.